Development Roadmap

Team: Karissa Kocourek and Daniel Escalante

Deadline: Friday, October 9 at 11:59 PM

Submission: Canvas (one submission per group)

In the previous milestone, you broke down your AI system's overall objective into a set of core AI modules and designed the flow of information based on a hypothetical version of your system that was already being used. Now, you will propose a plan for building each of these AI modules.

In this milestone, your team will develop a high-level plan for developing your system, focusing on the sequence of major steps and deliverables necessary to build your final AI modules. You will expand on this high-level plan in a follow-up milestone. At this stage, specific details about model architecture, training, evaluation, and experiment design are not yet expected.

The main objectives of this assignment are to:

- Develop an overall plan for how each AI module will be developed.
- Identify the datasets that you intend to use.
- Develop a tentative timeline and roadmap of key milestones on the path towards your team's first working prototype.

1. AI Design Project Update (1-3 sentences)

Provide a brief overall update on your project since the last milestone submission. Has anything changed in the overall problem, direction, or scope? Have you made any progress since the last project update? If there is nothing to report, type *N/A*.

AI Design Project Update:

Since Milestone 2, we inspected the open MIMIC-IV Clinical Database Demo. Across its 1,336 microbiology specimens, it confirmed that public hospital data records only specimen collection and the final result, with no events in between, so the modules will be developed on simulated specimen data. We also set up the project repository and documented the full system design.

2. AI Module Development

Note: This section will add detail to the AI modules that your team already submitted in the previous AI Solution Architecture milestone. If you've made any changes to your AI modules since submission, use the latest version.

Each of your AI modules from the previous milestone was centered around a distinct AI task. In most circumstances, an **AI model** will be the mechanism for accomplishing an AI task.

AI models can take many forms:

- "Traditional" machine learning models, e.g., logistic regression, support vector machine, random forest, XGBoost
- Deep learning models, e.g., convolutional neural network (CNN), recurrent neural network (RNN), multilayer perceptrons, etc.
- Open-source, open-weight, and/or local large language models (LLMs), e.g., Meta Muse Glimmer, NVIDIA Nemotron, Google Gemma, OpenAI GPT-OSS, etc.
- Frontier LLMs available in the cloud, e.g., Claude, ChatGPT, Gemini, etc.

Note: You do not need to identify or name any specific model or architecture in this milestone. The examples above are simply intended to illustrate the breadth of the AI model landscape.

How you modify and build with an AI model will also depend on its form. For some models, you'll be working in code; for others, you might develop via prompting. Sometimes models are already pre-trained (and either ready to use as-is or requiring fine-tuning). Other times you might need to develop your own model using a particular model architecture and/or algorithm.

The table below lists a few common approaches to working with AI models:

Modeling Approach	Description	Required Training Data	Example
Use an existing pre-trained model	Identify an existing model that has already been trained to perform the AI task you need for your AI module and use it as-is without any changes. Does not modify internal model weights.	No training dataset required.	Transcribe patient voicemails using an off-the-shelf speech-to-text model.
Fine-tune an existing model	Starting with an existing pre-trained model, continue training it on examples from your own dataset to allow the model to specialize in your task. Modifies internal model weights.	Requires a fine-tuning dataset for adapting the pre-trained model to a new domain (unlabeled fine-tuning data) or to a new task (labeled fine-tuning data). Recommended sample size is 100-10,000 examples.	Adapt a general-purpose (domain-agnostic) vision-language foundation model for assessing wound healing status from an image using a paired dataset of wound images and text descriptions.
Train a new model	Identify an appropriate model architecture for your task and train it from scratch on your own data. Modifies internal model weights (which start randomly initialized in an empty model.)	Requires a large dataset of (input, output) examples for the model to learn from. Recommended sample size heavily depends on the task, model, and data modality, but is often 1,000-1,000,000.	Build a model to predict a specific patient's 1-year hospital readmission probability, using a retrospective tabular dataset of patient information (e.g., age, reason for current visit, average time between

			past visits, etc.)
Context engineering (LLMs)	Include custom prompts and data as input to an existing LLM to control its output. Does not modify internal model weights.	No training dataset required (other than prompts). Optionally, you may also include data examples and/or reference documents to include in the prompt.	Prompt an LLM to generate standardized clinical notes for a given hospital by including the hospital's specific documentation guideline files in the prompt.
Harness engineering (agentic LLMs)	Design the execution layer of an agentic LLM, including tool orchestration, guardrails, feedback loops, etc.	No training dataset required. Can optionally include in-context domain examples, reference documents, and other files used to control agentic behavior.	Automate complex patient referrals with an agentic loop that can query patient databases, request missing documents, and escalate to specialists when needed.

Table 1. Example high-level approaches for working with AI models. This list shows some of the more common approaches, but your module might use an approach not listed in this table, including AI modules which don't use an underlying model, use multiple models, etc.

For each of your AI Modules, include the following information:

- **Module Description.** What does this module do? This can be taken directly from your AI Solution Architecture milestone assignment. If your team has made any changes since the previous submission, use the updated information.
- **AI Task.** What is the underlying AI task for this module? This can also be copied over from your AI Solution Architecture (include any changes).
- **Modeling Approach.** How will you use AI model(s) to achieve the task and overall module objective? For most modules, this will come from the first column of Table 1.
- **Development Plan.** Provide a brief, high-level description of how you will apply the modeling approach to achieve the module objective using the AI task.

An example row for one AI module is shown below:

Module Description	AI Task	Modeling Approach	Development Plan
Predict the probability that a patient will miss their next appointment.	Tabular Classification	Train a new model	Train a machine learning classifier to predict the probability of a patient missing their next appointment based on how many previous appointments they have missed in the past year.

Complete the table below with the AI Modules from your AI Solution Architecture. If your team has made any changes to the modules after submitting the previous milestone assignment, use the updated information.

AI Modules:

Module Description	AI Task	Modeling Approach	Development Plan
Specimen status: judges whether a specimen between scans is on track, delayed, or likely lost or stalled	Tabular classification	Train a new model	Train on simulated specimen histories where each specimen's true outcome is known. The model learns how long each leg normally takes for each specimen type, time of day, and lab volume, and outputs how likely a specimen is to be on track, delayed, or lost or stalled.
Arrival forecast: predicts when the next scan should occur	Tabular regression	Train a new model	Train on the same simulated histories, sharing the leg durations the status module learns, to predict when the next scan should occur with a likely and an outer bound.
Location estimate: infers where an unscanned specimen most likely is	Multiclass tabular classification	Train a new model	Train on simulated histories in which the true location of every specimen is recorded at every moment. The model uses tube-system events and elapsed time to output a probability for each possible location.
Search brief: tells the tech where to look first	Text-to-text generation	Context engineering with an LLM	Prompt an existing LLM with the outputs of the other three modules, the specimen details, and the stability reference table. A fixed template limits the note to where to look first, why, and how much time remains. No training data is required.

3. Datasets (1-3 datasets)

Data is the foundation of artificial intelligence, and high-quality datasets are crucial for developing effective AI applications. If you plan to train or fine-tune a model, choosing the right dataset is also one of the most important decisions in your project. In this section, you will identify 1-3 datasets that you intend to use to build or test your AI system.

A dataset can be a collection of patient records, notes, images, signals, or other data from biomedical and health science domains. It can also be a catalogue or repository of reference documents or external information that your system will use. The datasets you select should be based on your AI module development plan. If you aren't sure whether something is considered a dataset, reach out to the instructor.

All datasets must be publicly available, deidentified, synthetic, or otherwise approved by the instructor (see *Responsible Data Use* in the AI Design Project Guidelines). Some public datasets require registration, training, and/or credentialing before you can download them. All members of your team must be authorized to use the datasets you select.

If your project requires a specific type of dataset that does not exist online, you may generate a synthetic dataset that has similar properties to real-world data. However, this should be a last

resort. If you plan to generate synthetic data, list it as a dataset and briefly describe what you will be generating and the overall generation approach.

For each dataset, include the following:

- **Dataset.** Provide the dataset name (if it has one) and a hyperlink for accessing it online (see example below.) Do not type out the full URL inside the table.
- **Overview.** Briefly describe what the dataset contains. Include the number of samples, type of data and/or variables, outcomes, and any other relevant information.
- **Intended Use.** Describe how you intend to use the dataset, including the AI module(s) in which it will be used.

An example row for one dataset is shown below:

Dataset	Overview	Intended Use
eICU Collaborative Research Database (link)	200,859 ICU stays among 139,367 patients at 208 U.S. hospitals between 2014-2015. Variables include demographics, vital signs, laboratory tests, diagnoses, medications, and treatments. Outcomes include discharge disposition and in-hospital mortality (9% rate).	Train and test the classification model in the Mortality Risk Estimation module. The generalization of the model will be externally validated using multiple test sets from different source hospitals.

Datasets (1-3 datasets):

Dataset	Overview	Intended Use
Simulated Specimen Event Dataset	Planned size: 30 seeded runs of 7 simulated days, about 50,000 specimens. Five specimen types: urine, body fluid, bronchoalveolar lavage, culture swab, and tissue, each with its ordered tests, including anaerobic culture. Each record has timestamps for collection, tube dispatch and arrival, the wait on the lab counter, receipt, setup, and plating, plus the true location at every moment and outcome labels. Injected failures: lost in tube storage, stalled on the counter, misrouted to another station, and never sent. Generated by a simulation of one collecting area, one pneumatic tube system, and one microbiology laboratory. Stability windows come from the reference table below. All timing distributions are assumed and labeled as such, and none come from any institution's records. The generator code will be published in the project repository.	Train and test the specimen status, arrival forecast, and location estimate modules, and power the interactive demo.
Specimen Stability Reference Table (link)	The stability window and transport condition for each of the five specimen types and each test ordered on them, one row per specimen type and test, each with its citation. Compiled from the IDSA and ASM 2024 Guide to Utilization of the Microbiology Laboratory for Diagnosis of Infectious Diseases. Transport swab manufacturer instructions serve as a secondary source for swab hold times (link).	Sets the effective window behind the time remaining before recollection, which is the minimum across the specimen type and every test ordered on it. That value feeds the search brief and alert ranking. The table also parameterizes the simulator.

Dataset	Overview	Intended Use
MIMIC-IV Clinical Database Demo v2.2 (link)	100 critical care patients with 1,336 microbiology specimens. The microbiology table records collection time and final result time only, a median of 80.1 hours apart, with no transport, receipt, or setup events.	Not used to train any module. It is used to test that the pipeline reads real microbiology records, and it serves as measured evidence that existing hospital data does not capture the interval Tracer monitors. That gap is why simulated data is the training source.

4. Development Roadmap

Starting from any progress your team has already made, outline a tentative roadmap and timeline of 3-5 major stages of development towards building a functional AI system prototype. Stages should represent distinct high-level periods of development which include related work.

Each team's development roadmap will look different. The stages of your roadmap can align with completion of your AI models and/or modules, or they may coincide with similar types of work (e.g., a stage for training multiple prediction models across different AI modules). Your roadmap will depend on how your team decides to break down and distribute smaller tasks based on the overall objective of your project. Roadmaps will almost certainly change as you progress further into the development of your system based on real-world constraints, feasibility, or changes in direction.

For each major stage of development, include the following:

- **Stage.** A name which summarizes this stage of project development.
- **Target End Date.** An approximate date which you expect to complete this stage of development.
- **Activities.** A brief description of what you will be doing during this stage of development.
- **Deliverables.** Describe the concrete outcome of this stage.

An example row for one stage of development is shown below:

Stage	Target End Date	Activities	Deliverables
Prepare Datasets	October 1	We will download all 3 imaging datasets (1 X-ray, 1 CT, and 1 MRI). We will filter each dataset to keep only images for patients who underwent major surgery. We will also link each patient's image with a tabular dataset of demographic variables and 1-year mortality status.	Final datasets for training prediction models (3 imaging datasets, 1 tabular dataset)

Development Roadmap (3-5 stages):

Stage	Target End Date	Activities	Deliverables
Specimen policy and workflow specifi-cation	October 16	Karissa compiles published stability and transport requirements for the five specimen types and the anaerobic culture rule, and confirms the workflow sequence and failure modes. Daniel encodes them as configuration.	A sourced stability table and a confirmed workflow map
Simulated specimen dataset	October 23	Daniel builds the simulator and generates event histories with injected failures. Karissa reviews sample specimen timelines for realism.	The simulated dataset with ground-truth location and outcomes
Status and arrival forecast	October 30	Daniel develops the specimen status and arrival forecast modules and connects them to a live board that ranks simulated specimens and raises alerts. Karissa reviews a sample of alerts and sets how much warning staff need to recover a specimen.	The minimum viable prototype
Location estimate and search brief	November 13	Daniel develops the location module and the search brief prompt, and connects all four modules into one end-to-end run. Karissa reviews sample search briefs for clinical accuracy and confirms who each alert is routed to.	A complete single run producing an alert with a search brief
Interactive demo tutorial	December 2	We build the public demo on simulated data. Karissa writes the clinical background and the workflow narrative, and Daniel builds the interactive walkthrough.	The published demo and the GitHub repository